AWS IAM from the Ground Up

Users, roles, and how the cloud-siem-honeypot project uses them

1	The basics: who, what, which resource
2	Policies: the permission documents
3	IAM users
4	IAM roles
5	Where your code uses roles (line by line)
6	Why there are no keys in your code: the credential chain
7	How to decide: role, user, or something else
8	Wazuh: why it got a user, and better options
9	Ways to run Wazuh
10	Cheat sheet & self-check questions

1. The basics: who, what, which resource

Every call to AWS ("put this log line", "write this S3 object", "run this Athena query") is a signed HTTP request. When it arrives, AWS IAM answers two questions:

Question	Name	How AWS answers it
Who is calling?	Authentication	It checks the request's cryptographic signature, which was made with an access key
Are they allowed to do this action on this resource?	Authorization	It reads the policies that apply to the caller and to the resource

The thing making the call is called a **principal**. In this project the principals are:

- **You**, running `terraform apply` with the keys from `aws configure`
- The **honeypot EC2 instance**, through the CloudWatch Agent
- The **Lambda function**, writing to S3
- The **CloudWatch Logs service**, invoking your Lambda
- **Wazuh** on your laptop, reading CloudWatch (Phase 5)

IAM's building blocks

Thing	What it is	Has its own credentials?
Root user	The email you signed up with. Can do everything. Use it only for account recovery and billing	Yes (password)
IAM user	A permanent identity for one person or one program	Yes: password and/or access keys
IAM group	A bucket of users that share policies. It can't log in itself	No
IAM role	An identity that someone or something *temporarily puts on*	**No**: you get temporary keys when you assume it
Policy	A JSON document listing allowed or denied actions	Not an identity

> **The three rules of policy evaluation**
>
> - **Default deny.** If nothing says Allow, the answer is no.
> - **An explicit Allow** in an applicable policy turns it into a yes.
> - **An explicit Deny always wins**, even over an Allow.

2. Policies: the permission documents

A policy is JSON. Each **statement** says: *Effect* (Allow/Deny) these *Actions* on these *Resources*, optionally only if a *Condition* is true. Here is one from your own project, the Lambda's S3 permission:

terraform/phase3.tf : 42-53

```
resource "aws_iam_role_policy" "lambda_s3_write" {
  role = aws_iam_role.lambda_exec.id            # <- WHO this applies to
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect   = "Allow"                        # Allow or Deny
      Action   = ["s3:PutObject"]               # WHAT: service:Operation
      Resource = "${aws_s3_bucket.datalake.arn}/*"  # ON WHAT: an ARN
    }]
  })
}
```

This is least privilege in practice: the Lambda can **only** create objects, **only** in that one bucket. It can't read, delete or list anything. If someone stole its credentials, the worst they could do is upload junk.

Two places a policy can live

Type	Attached to	Answers	Example in your project
Identity-based	A user, group or role	"What can *this identity* do?"	`lambda_s3_write`, `CloudWatchAgentServerPolicy`, `wazuh_cloudwatch_read`
Resource-based	A resource (Lambda, S3 bucket, a role's trust policy...)	"Who may touch *this resource*?"	`aws_lambda_permission` (CloudWatch may invoke the Lambda), and every role's `assume_role_policy`

Three ways to write identity-based policies

Kind	Terraform resource	When to use
AWS-managed	`policy_arn = "arn:aws:iam::aws:policy/..."`	Fast to set up, but often broader than you need. Example: `CloudWatchAgentServerPolicy` (main.tf:113)
Customer-managed	`aws_iam_policy`	Reusable: write it once and attach it to many identities. Example: `athena_analyst` (phase4.tf:143). Note: it isn't attached to anyone yet, so it does nothing until it is
Inline	`aws_iam_role_policy` / `aws_iam_user_policy`	Belongs to exactly one identity and is deleted along with it. Examples: `lambda_s3_write`, `wazuh_cloudwatch_read`

3. IAM users

An IAM user is a **permanent identity**. Think of it as an employee ID badge: it's issued once and works until someone revokes it. A user can have:

- **A console password**, for a human logging into the web console (add MFA)
- **Access keys**, for programs: an access key ID starting with `AKIA` plus a secret access key

The problem with access keys

Access keys are **long-term**. They never expire on their own. Anyone who copies the two strings *is* that user, from anywhere in the world, until you delete the key. That's how most AWS breaches happen:

- Keys committed to GitHub. Bots scan every public push within minutes, much like the bots hitting your honeypot
- Keys left in a laptop backup, a Docker image layer, a screenshot, or a CI log
- Keys kept by someone who left the team, because nobody remembered to delete them

> **Where you already use an IAM user**
> When you ran `aws configure` and pasted an access key, you created a local file `~/.aws/credentials`. Terraform reads it to act as **you**. That's a normal, accepted use of an IAM user: a human at their own machine. (If that key belongs to the **root** user, create an admin IAM user with MFA and delete the root keys.)

Good practice for users that must exist

- Give them the fewest permissions possible (Wazuh's user can only read one log group)
- Rotate keys regularly, and never have more than one active key per user
- Restrict where they can be used from with a `Condition` such as `aws:SourceIp`
- Prefer a user whose *only* permission is assuming a role (section 8)

4. IAM roles

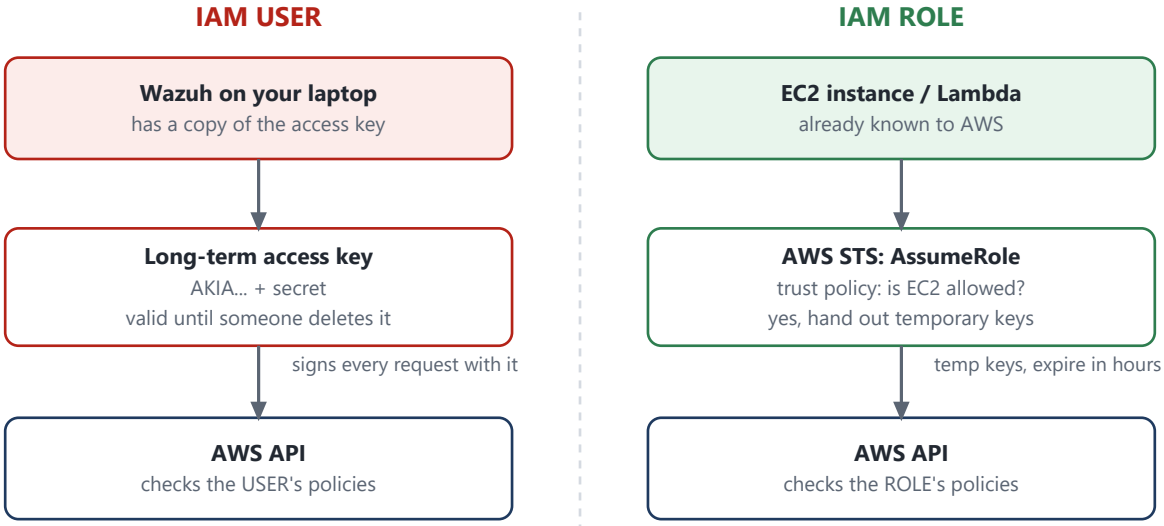
A role is an identity **without credentials of its own**. Instead of a badge, think of a visitor pass at a reception desk: you prove who you are, the desk checks the guest list, and you get a pass that **expires at the end of the day**.

Every role has two policies, and the difference matters:

	Trust policy	Permission policies
Question	**Who** may assume (put on) this role?	**What** can whoever wears it do?
Terraform	`assume_role_policy` inside `aws_iam_role`	`aws_iam_role_policy` or `aws_iam_role_policy_attachment`
Honeypot example	Principal `ec2.amazonaws.com` (main.tf:104)	`CloudWatchAgentServerPolicy` (main.tf:111-113)
Lambda example	Principal `lambda.amazonaws.com` (phase3.tf:30)	Basic execution logs + `s3:PutObject` (phase3.tf:36-53)

Assuming a role

When an allowed principal assumes a role, **AWS STS** (Security Token Service) returns three values: an access key ID starting with `ASIA` (temporary), a secret, and a **session token**. They expire after anything from 15 minutes to 12 hours, depending on how they were requested. For EC2 and Lambda, AWS refreshes them automatically, so you never handle them.



Why roles are safer

- **Nothing to leak in your code or Git:** there are no keys in any file
- **A stolen key goes stale quickly:** temporary credentials expire within hours
- **The trust policy** limits who can even attempt to use the role
- **CloudTrail** logs every AssumeRole call, so you can see who used it and when

5. Where your code uses roles (line by line)

A role on its own does nothing. Something has to **connect** it to the compute that will wear it. This chain is what tells AWS "this EC2 instance / this Lambda uses this role".

5.1 The honeypot EC2: four links in a chain

terraform/main.tf

```
# (1) The role + TRUST policy: "EC2 instances may wear this"
resource "aws_iam_role" "honeypot_role" {                        # main.tf:94
  name = "honeypot_cloudwatch_role"
  assume_role_policy = jsonencode({ ...
        Principal = { Service = "ec2.amazonaws.com" }            # main.tf:104
  })
}

# (2) PERMISSIONS: what the wearer can do
resource "aws_iam_role_policy_attachment" "cloudwatch_agent_policy" {  # main.tf:111
  role       = aws_iam_role.honeypot_role.name
  policy_arn = "arn:aws:iam::aws:policy/CloudWatchAgentServerPolicy"
}

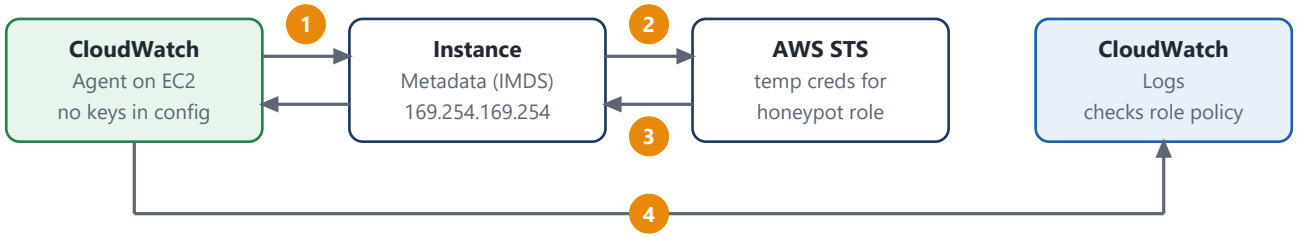
# (3) INSTANCE PROFILE: the "holder" EC2 needs to carry a role
resource "aws_iam_instance_profile" "honeypot_profile" {          # main.tf:116
  role = aws_iam_role.honeypot_role.name
}

# (4) The link that makes the instance actually USE it
resource "aws_instance" "honeypot" {
  ...
  iam_instance_profile = aws_iam_instance_profile.honeypot_profile.name  # main.tf:136
}
```

Why does EC2 need an "instance profile"?

That's simply how EC2 was designed: an instance can't hold a role directly, only a profile that contains one role. The AWS console creates the profile for you without telling you; Terraform makes you do it explicitly. Lambda, ECS and the others don't need one. They take the role ARN directly.

What happens at runtime when your CloudWatch Agent ships a Cowrie log line



1 Agent asks the metadata service: "which credentials do I have?"
2 EC2 gets temporary credentials from STS for the instance profile's role (and keeps them fresh)
3 Temporary keys (ASIA... + token) are returned to the agent
4 Agent calls PutLogEvents signed with those keys; IAM checks the role's policy

5.2 The Lambda: one link

terraform/phase3.tf

```
resource "aws_iam_role" "lambda_exec" {                          # phase3.tf:23
  assume_role_policy = jsonencode({ ...
      Principal = { Service = "lambda.amazonaws.com" }           # phase3.tf:30
  })
}

resource "aws_lambda_function" "normalizer" {
  ...
  role = aws_iam_role.lambda_exec.arn     # phase3.tf:66  <- THIS line is the link
}
```

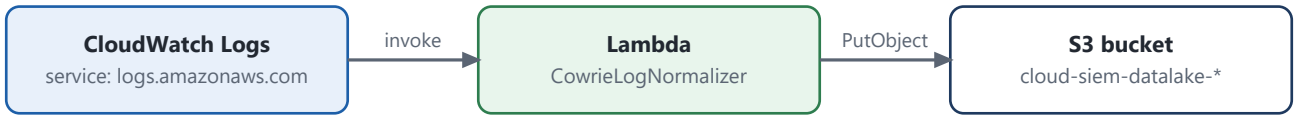
5.3 The third mechanism: resource-based permission

CloudWatch Logs calling your Lambda doesn't use a role at all. Instead, the **Lambda itself** carries a resource-based policy saying who may invoke it:

terraform/phase3.tf

```
resource "aws_lambda_permission" "cloudwatch_invoke" {           # phase3.tf:81
  action        = "lambda:InvokeFunction"
  principal     = "logs.amazonaws.com"                           # WHO may call
  source_arn    = "${aws_cloudwatch_log_group.cowrie_logs.arn}:*" # ...only from THIS log group
}
```

`source_arn` matters. Without it, *any* CloudWatch log group, including one in someone else's account, could trigger your function. This is called the **confused deputy** problem.



Resource-based policy ON the Lambda
`aws_lambda_permission (phase3.tf:81)`

Execution ROLE of the Lambda
`aws_iam_role.lambda_exec (phase3.tf:23)`

Incoming calls are controlled by the target's resource policy. Outgoing calls use the caller's role.

6. Why there are no keys in your code: the credential chain

Look at your Lambda:

scripts/lambda_function.py

```
import boto3
s3_client = boto3.client('s3')      # lambda_function.py:8  - no keys anywhere!
```

So how does it authenticate? Every AWS SDK (boto3, the CLI, the CloudWatch Agent, Terraform) searches for credentials in a **fixed order** and uses the first one it finds:

#	Where the SDK looks	Who puts credentials there
1	Keys passed explicitly in code (`boto3.client('s3', aws_access_key_id=...)`)	Nobody, ideally. **Never do this**
2	Environment variables `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_SESSION_TOKEN`	**Lambda** sets these automatically from its execution role
3	Shared files `~/.aws/credentials` and `~/.aws/config` (profiles)	**You**, via `aws configure`. Also Wazuh's mounted credentials file
4	Container credentials endpoint	ECS / EKS tasks with a task role
5	EC2 Instance Metadata Service `169.254.169.254`	**EC2**, from the instance profile. This is what your CloudWatch Agent uses

The key insight

The code never says "use the role". The role is chosen by the *infrastructure* (Terraform), and the SDK finds its temporary credentials automatically. That's why your `setup_cloudwatch.sh` and `lambda_function.py` contain no secrets, and why the same Python code works on your laptop (it finds your profile, #3) and in Lambda (it finds the env vars, #2) without changes.

Security note for a honeypot

Any process on the EC2 instance can call `169.254.169.254` and get the role's keys. Cowrie only *emulates* a shell, so attackers shouldn't reach the real OS. But if they ever did, they'd get this role. That's why it must only have log-writing permissions. Two improvements: (a) require IMDSv2 with `metadata_options { http_tokens = "required" }` on `aws_instance`, which blocks a common way of stealing these credentials; (b) `CloudWatchAgentServerPolicy` is broader than "write only" (it also allows metric writes and some EC2/SSM reads). A custom policy allowing just `logs:CreateLogStream` / `logs:PutLogEvents` on `/honeypot/cowrie` is tighter.

7. How to decide: role, user, or something else

Ask these questions **in order**. Stop at the first "yes".

#	Question	If yes, use	Example
1	Is it **code running on AWS compute**? (EC2, Lambda, ECS, EKS, CodeBuild, Glue job...)	**Role** attached to that compute	Honeypot EC2, CowrieLogNormalizer
2	Is it an **AWS service calling another of your resources**?	**Resource-based policy** naming the service principal (plus `source_arn`)	CloudWatch Logs invoking Lambda
3	Is it **outside AWS but supports OIDC/web identity**? (GitHub Actions, GitLab CI, other clouds, Kubernetes)	**Role** with an OIDC trust policy. No keys at all	Phase 8 CI running `terraform plan`
4	Is it **outside AWS and you can give it a certificate**?	**Role** via IAM Roles Anywhere	Wazuh on a server you manage
5	Is it a **human**?	IAM Identity Center (SSO) in organisations; in a personal lab, an **IAM user + MFA**	You running Terraform
6	None of the above: a program outside AWS that can only use static keys	**IAM user** with minimal permissions, or better a user that can only assume a role	Wazuh on your laptop (today)

Your project, mapped

Component	Runs where	Identity	Why
Terraform (you)	Your laptop	IAM user (your keys)	Human, personal lab (rule 5)
CloudWatch Agent	EC2	Role via instance profile	AWS compute (rule 1)
CowrieLogNormalizer	Lambda	Execution role	AWS compute (rule 1)
CloudWatch → Lambda	AWS service	Resource policy on Lambda	Service-to-resource (rule 2)
Athena analyst	Console / CLI	`athena_analyst` policy, to attach to a user or role	Reusable permission set
Wazuh	Your laptop (Docker)	IAM user (Phase 5)	Outside AWS, static keys only (rule 6)

> **Rule of thumb**
> **Roles for workloads, users only as a last resort.** If you're about to create an access key, stop and check whether rules 1–4 apply first.

8. Wazuh: why it got a user, and better options

Wazuh in Docker on your laptop fails rules 1–3: it isn't AWS compute, it's not an AWS service, and it doesn't have an OIDC identity provider. AWS has no way to recognise your laptop, so there's nothing it could use to assume a role. It needs **some** starting secret. That's why `phase5.tf` creates an IAM user.

Option	How it works	Pros	Cons
A. User + access key (current `phase5.tf`)	The user has read-only CloudWatch permissions directly	Simplest: 3 Terraform resources	The key never expires; a leak exposes your logs until it's rotated
B. User that can only assume a role	The user's only permission is `sts:AssumeRole`. Wazuh assumes the role and gets temporary keys	The key alone can't read anything; you can lock it to your IP; CloudTrail shows each session	The key itself is still long-lived
C. IAM Roles Anywhere	Laptop proves its identity with an X.509 certificate and gets temporary role keys	No long-lived AWS keys at all	You must run a small certificate authority; more setup
D. Run Wazuh on EC2	Instance profile, exactly like the honeypot	No keys; the cleanest option	Costs ~$60/month while running

What option B looks like

Option B (sketch for phase5.tf)

```
# Role holds the real permissions
resource "aws_iam_role" "wazuh_reader" {
  name = "wazuh-cloudwatch-reader-role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect    = "Allow"
      Action    = "sts:AssumeRole"
      Principal = { AWS = aws_iam_user.wazuh_reader.arn }      # only this user
      Condition = { IpAddress = { "aws:SourceIp" = var.your_ip } }  # only from home
    }]
  })
}
# -> move the logs:Describe*/GetLogEvents/FilterLogEvents statements onto this role

# The user is allowed to do exactly ONE thing
resource "aws_iam_user_policy" "wazuh_assume_only" {
  user   = aws_iam_user.wazuh_reader.name
  policy = jsonencode({ Version = "2012-10-17", Statement = [{
    Effect = "Allow", Action = "sts:AssumeRole", Resource = aws_iam_role.wazuh_reader.arn
  }]})
}
```

Wazuh config change for option B

```
<service type="cloudwatchlogs">
  <aws_profile>wazuh</aws_profile>
  <iam_role_arn>arn:aws:iam::<ACCOUNT_ID>:role/wazuh-cloudwatch-reader-role</iam_role_arn>
  <aws_log_groups>/honeypot/cowrie</aws_log_groups>
  <regions>us-east-1</regions>
</service>
```

9. Ways to run Wazuh

Wazuh has three central parts: the **manager** (runs rules), the **indexer** (stores alerts; it's OpenSearch) and the **dashboard** (web UI). Together they need about 4 vCPU and 8 GB RAM. Where you run them affects cost, uptime and which IAM identity you can use.

Option	Cost	Uptime	AWS identity	Best for
Docker single-node on your laptop	Free	Only while the laptop is on (it catches up from CloudWatch when restarted)	IAM user (A/B) or Roles Anywhere (C)	**Learning: start here**
Linux VM (VirtualBox/VMware) with the all-in-one install script or Wazuh's OVA	Free	While the VM is on	Same as Docker	Learning how a real install is laid out (systemd services, `/var/ossec`)
EC2 t3.large+ (install script or Marketplace AMI)	~$60/month on, ~$2/month stopped (disk only)	24/7	**Instance role**: no keys	A "real" always-on SIEM for the portfolio
Docker multi-node / Kubernetes	$$	High availability	Role (EKS IRSA / pod identity)	Showing scaling skills; too much for this project
Wazuh Cloud (SaaS)	Paid after trial	Managed	Static keys	Zero ops, but you learn less

Agent or agentless?

Separately from *where* Wazuh runs is *how logs reach it*:

- **Pull from CloudWatch (what you're doing):** Wazuh's AWS module fetches logs over the AWS API. The honeypot has no idea Wazuh exists.
- **Wazuh agent on the honeypot:** the agent pushes logs to the manager on ports 1514/1515. It's faster (real-time) and adds file integrity monitoring, but it creates a **network path from the box you want attackers on to your SIEM**.

Recommendation

- **Now:** Docker on your laptop with option A or B. It's free and enough to learn rules, decoders and dashboards.
- **Later (optional):** move to EC2 in its own security group, dashboard limited to your IP, with an instance role. The access key problem disappears, and you stop the instance when you're not using it.
- **Keep the CloudWatch pull** rather than an agent. Mention the isolation reasoning in your threat model.

10. Cheat sheet

Term	One-line meaning
Principal	Whoever is making the request (user, role session, AWS service)
IAM user	Permanent identity with long-term credentials (password / AKIA key)
IAM role	Identity with no credentials; assumed for temporary ASIA keys + session token
Trust policy	On a role: who may assume it (`assume_role_policy`)
Permission policy	What an identity may do (Effect / Action / Resource / Condition)
Resource-based policy	On a resource: who may access it (`aws_lambda_permission`, bucket policy)
Instance profile	EC2-only wrapper that carries a role onto an instance
STS	Security Token Service: issues the temporary credentials
IMDS	`169.254.169.254`: where EC2 code fetches its role credentials
Credential chain	The fixed order in which SDKs look for credentials (code → env → files → container → IMDS)
Least privilege	Grant only the exact actions and resources needed, nothing more
Confused deputy	A service tricked into acting for the wrong party; prevented with `source_arn` / `aws:SourceAccount`

Self-check questions

Q1. Your Lambda has no keys in its code. How does boto3 authenticate?

Answer: Lambda injects the execution role's temporary credentials as environment variables, and boto3 finds them at step 2 of the credential chain.

Q2. What would break if you deleted line 136 of main.tf?

Answer: The instance would have no instance profile, so the CloudWatch Agent would find no credentials and fail to ship logs.

Q3. What's the difference between the role's trust policy and the CloudWatchAgentServerPolicy attachment?

Answer: The trust policy says who may wear the role (EC2). The attachment says what the wearer can do (write logs).

Q4. Why doesn't CloudWatch Logs need a role to invoke your Lambda?

Answer: The Lambda's resource-based policy (aws_lambda_permission) directly allows logs.amazonaws.com, limited to your log group by source_arn.

Q5. Someone finds the Wazuh access key in a screenshot. What can they do, and what do you do?

Answer: With option A, they can read the honeypot logs. Delete the key immediately (terraform apply -replace=aws_iam_access_key.wazuh_reader) and check CloudTrail. With option B plus an IP condition, they couldn't use it from their own network at all.

Q6. You add GitHub Actions to run terraform plan. User or role?

Answer: A role with an OIDC trust policy for token.actions.githubusercontent.com. No stored keys (rule 3).